

[NLP & Text Mining]

Word2Vec + Down-Stream Task

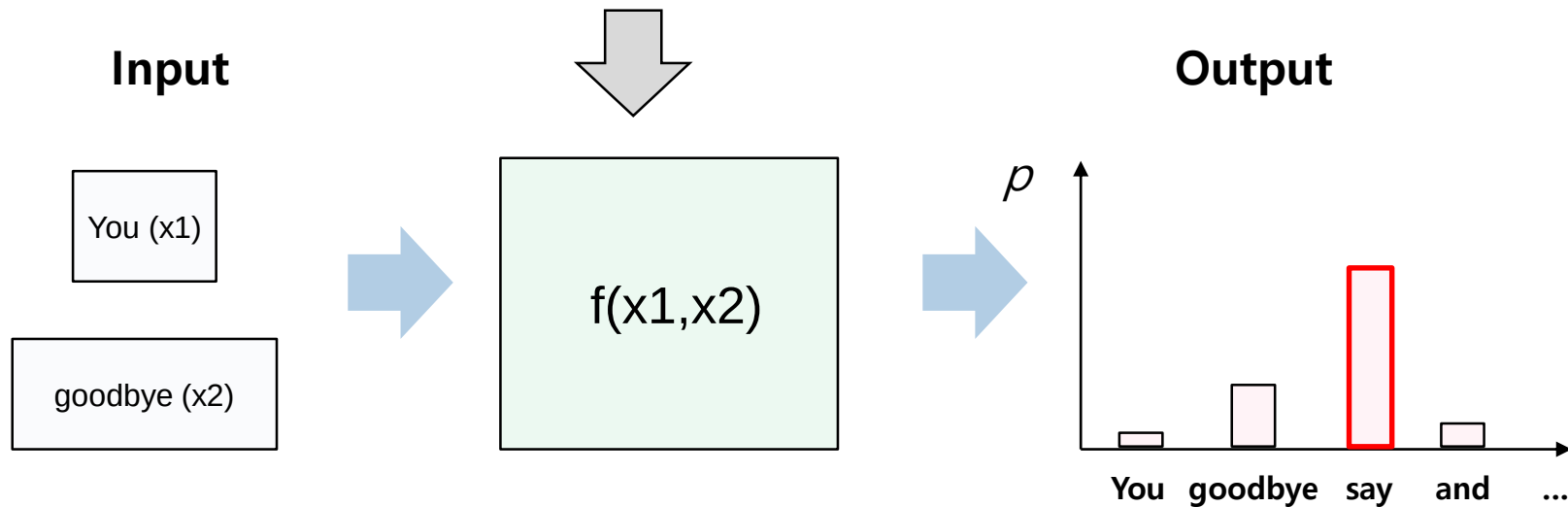
박성호

컴퓨터공학부

Word2Vec - CBOW



- 주변에 맥락을 Input으로 받아서 ? 부분에 적합한 단어를 Output으로 산출하는 NN 모델을 학습 및 활용
- 맥락정보를 받아서 각 단어의 출현 확률을 출력하는 NN

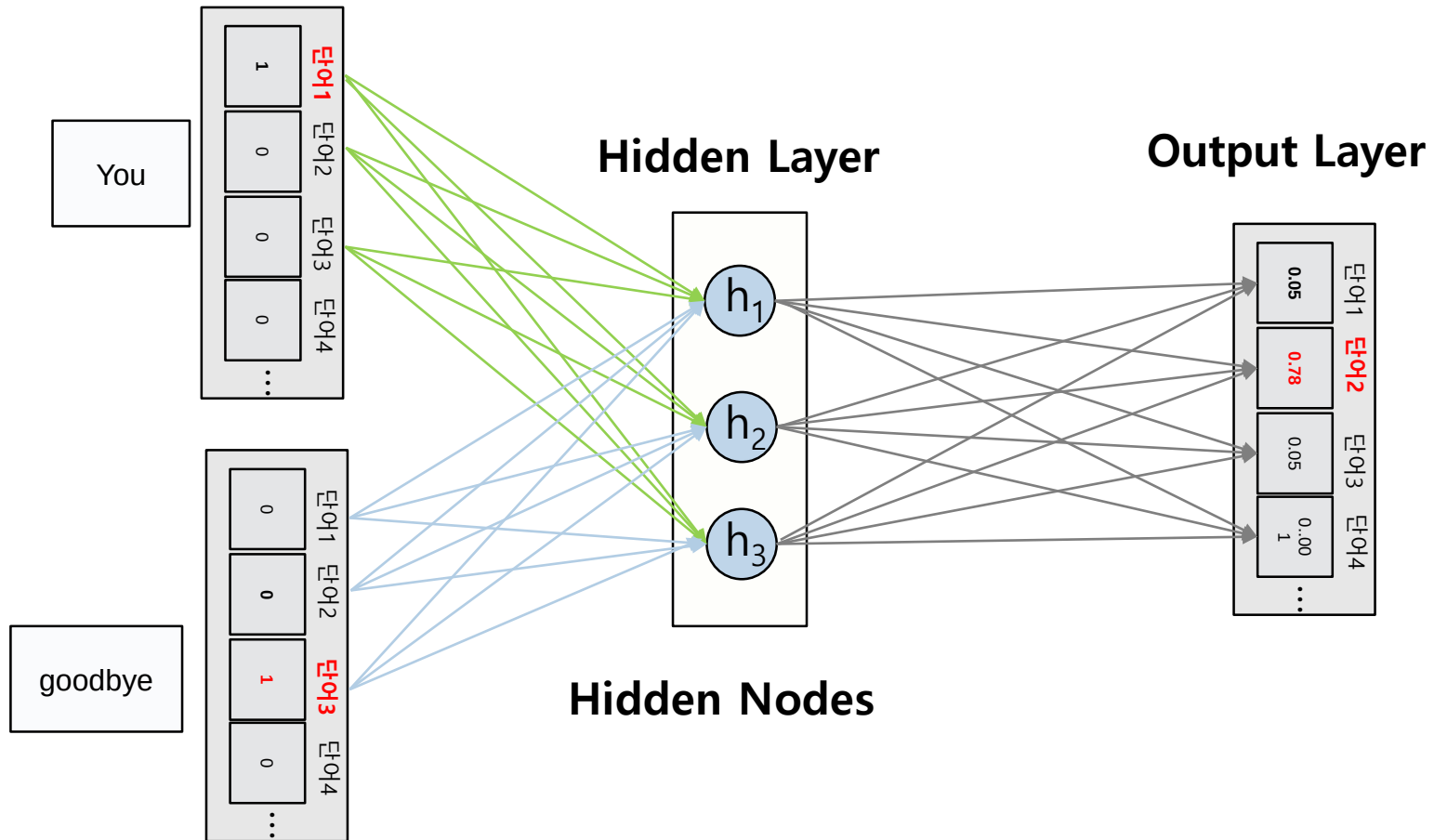


Word2Vec - CBOW

Step2: Word2Vec NN 학습

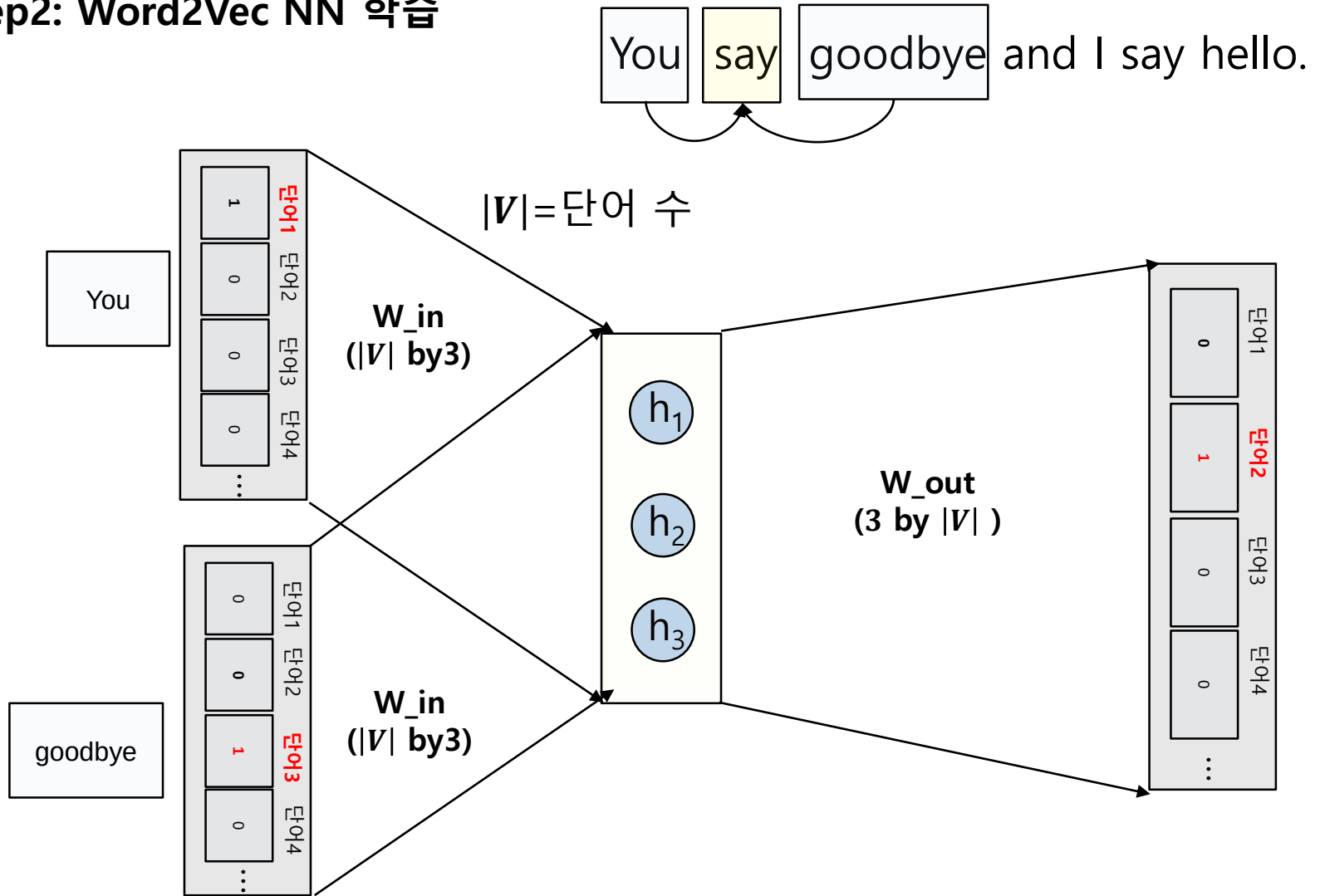
Input Layer

You say goodbye and I say hello.



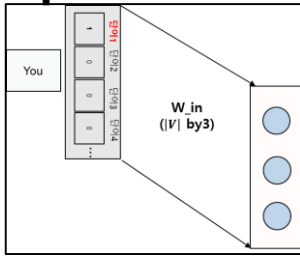
Word2Vec - CBOW

Step2: Word2Vec NN 학습



Word2Vec - CBOW

Step2: Word2Vec NN 학습



You say goodbye and I say hello.

W_{in}
($|V|$ by 3)

You

$|V|$ = 단어 수

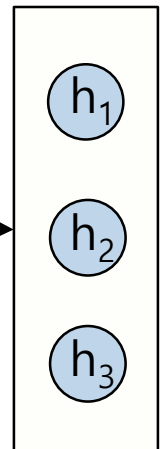


$\times$

$|V|$

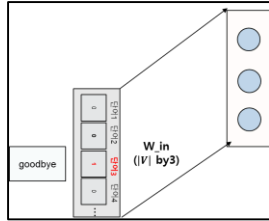
853. (W_{11})	5.2 (W_{12})	7.8 (W_{13})

3



Word2Vec - CBOW

Step2: Word2Vec NN 학습



You say goodbye and I say hello.

W_{in}
($|V|$ by 3)

goodbye

$|V|$

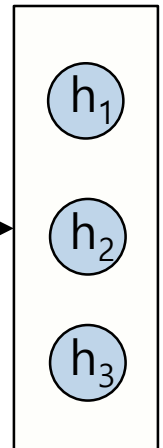


×

$|V|$

853. (W_{31})	5.2 (W_{32})	7.8 (W_{33})

3



Down-stream task (pre-trained 모델 활용)

Text Regression/Classification (분류, 감성분석..등등)

X (Input)

Y (Output)

Sentence1: You say goodbye and I say hello.

0(긍)

Sentence2: You said goodbye and I said go to hell.

1(부)

⋮



Sentence1: You say goodbye and I say hello. -> 임베딩1(벡터화)

Sentence2: You said goodbye and I said go to hell. -> 임베딩2(벡터화)

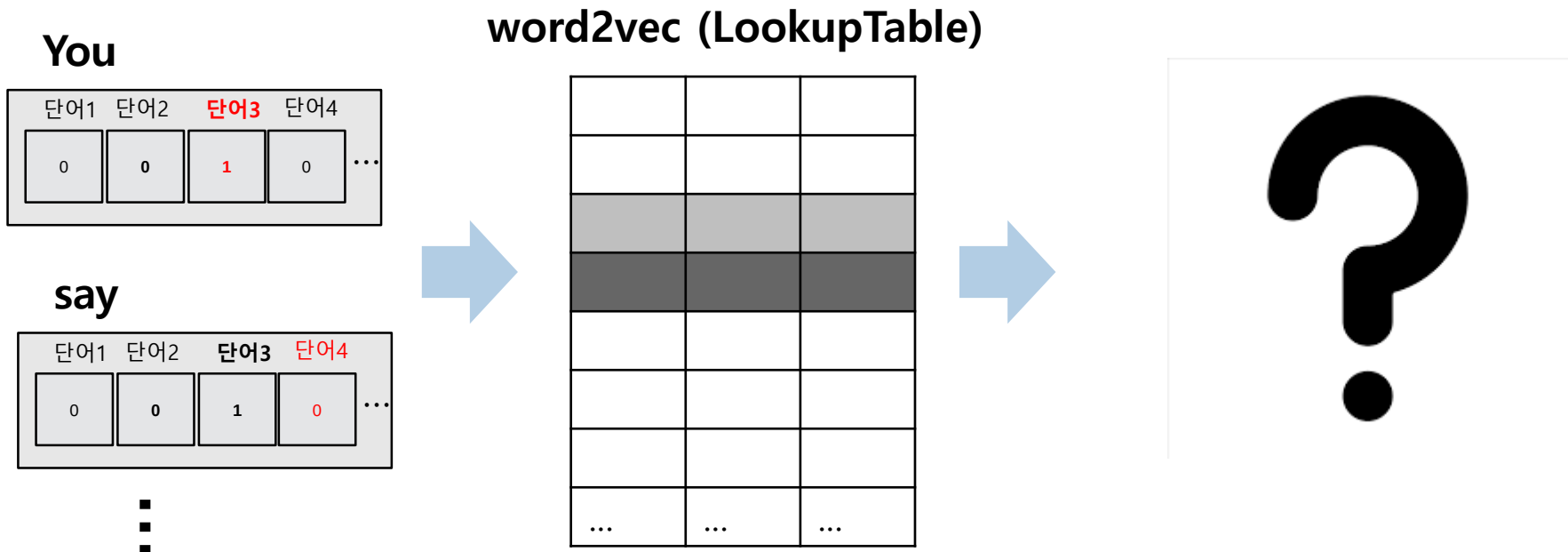
Word2Vec – Text

Sentence1: You say goodbye and I say hello.

Sentence2: You said goodbye and I said go to hell.

⋮

Sentence1: You say goodbye and I say hello. -> 임베딩(벡터화)



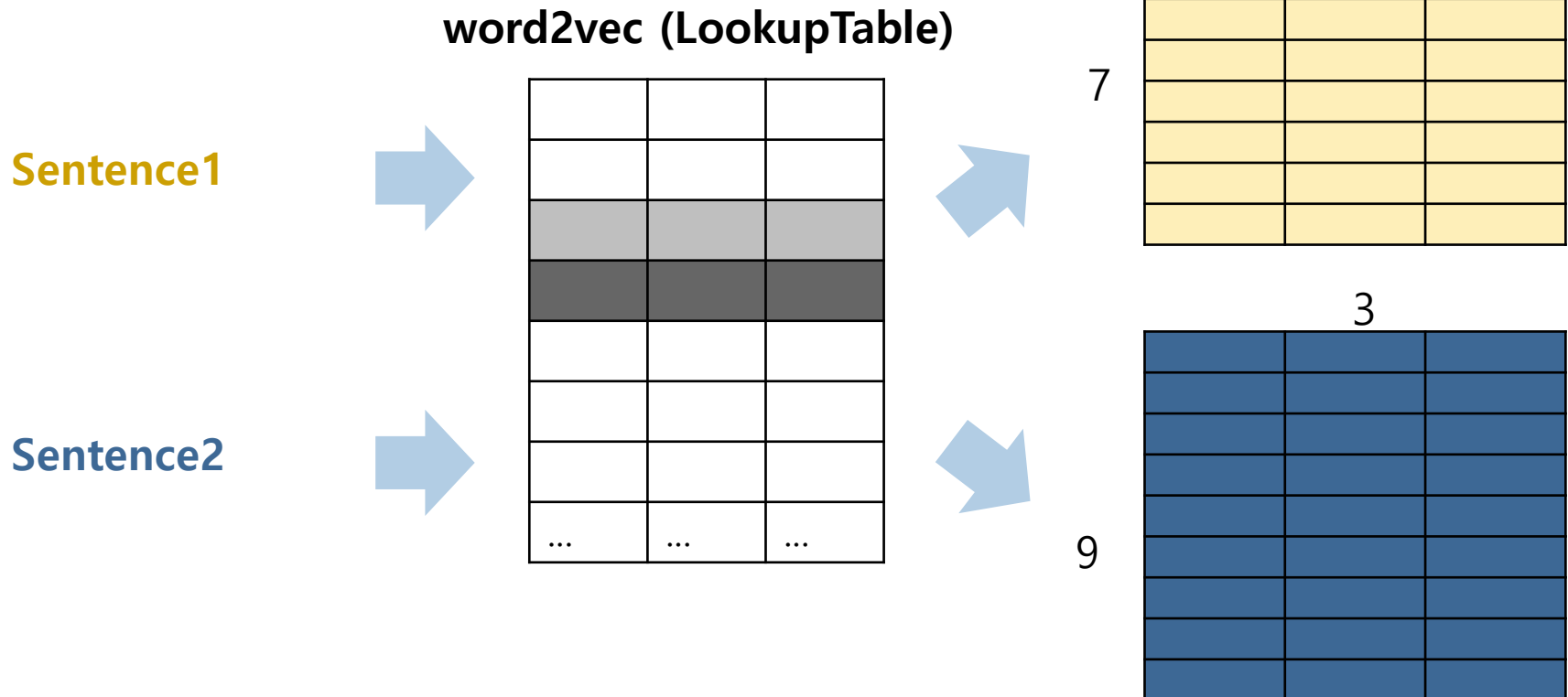
Word2Vec – Text

Sentence1: You say goodbye and I say hello.

Sentence2: You said goodbye and I said go to hell.

⋮

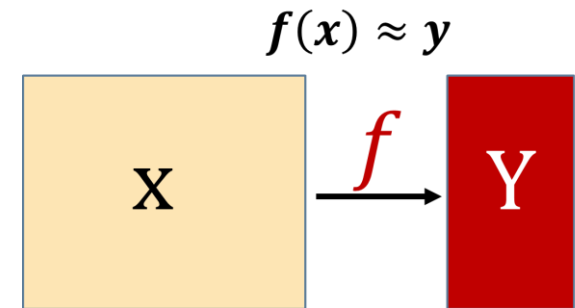
Sentence1: You say goodbye and I say hello. -> 임베딩(벡터화)



지도 학습 (Supervised Learning)

문서분류 분석-> Spam Email 분류

Term Variables	Documents			
Term 1	Document1 1	Document2	...	Document n
Term 2	Data			
⋮				
Term m				



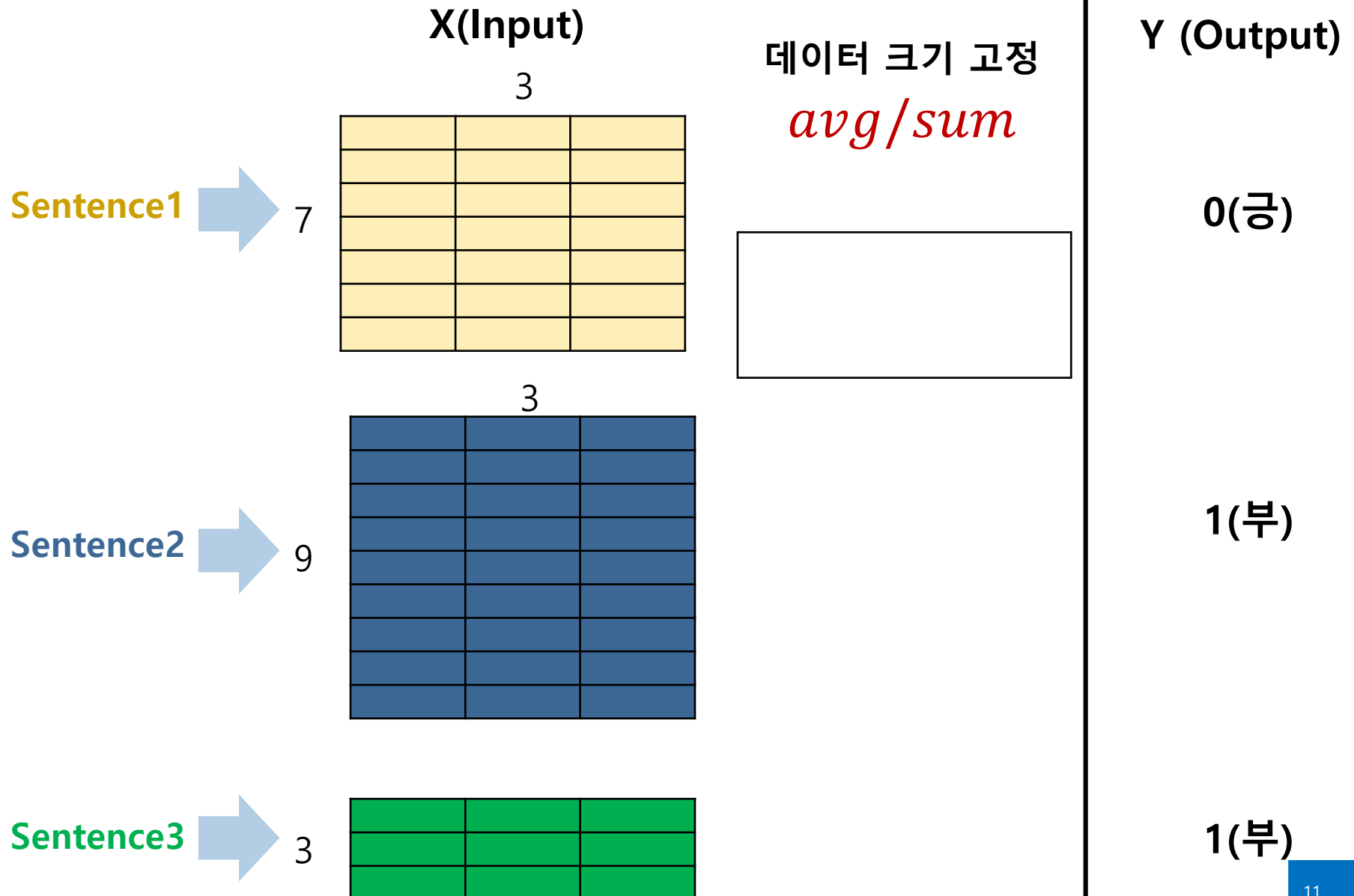
X

	Word1	Word2	Word3	Word4	Word5	...	Word k
Email1							
Email2							
Email3							
Email4							
Email5							
...							

y

y
SPAM
N
SPAM
SPAM
N
N

Word2Vec – Text



Word2Vec – Text

X(Input)

$$\mathbf{f}(\mathbf{x}) \approx \mathbf{y}$$

Y (Output)

Sentence1

Sentence2

Sentence3

Sentence4

Sentence5

•
•
•

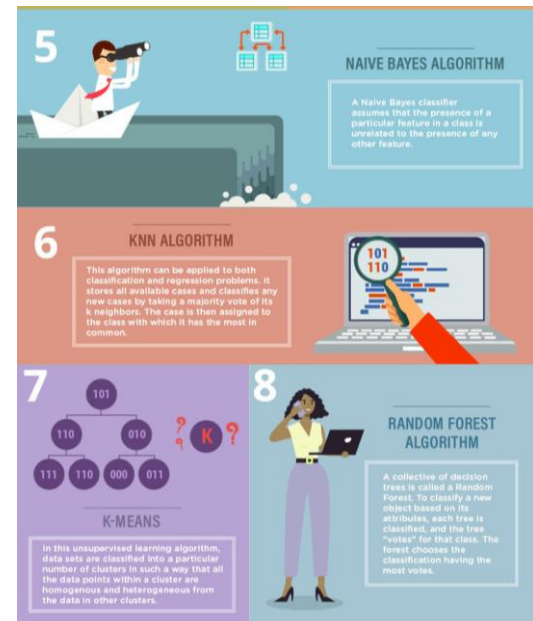
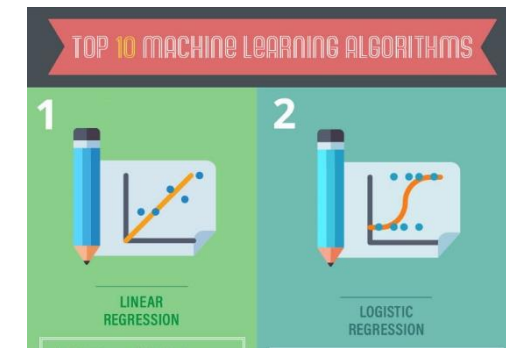
0

1

1

0

0



12

Neural Networks (CNN)

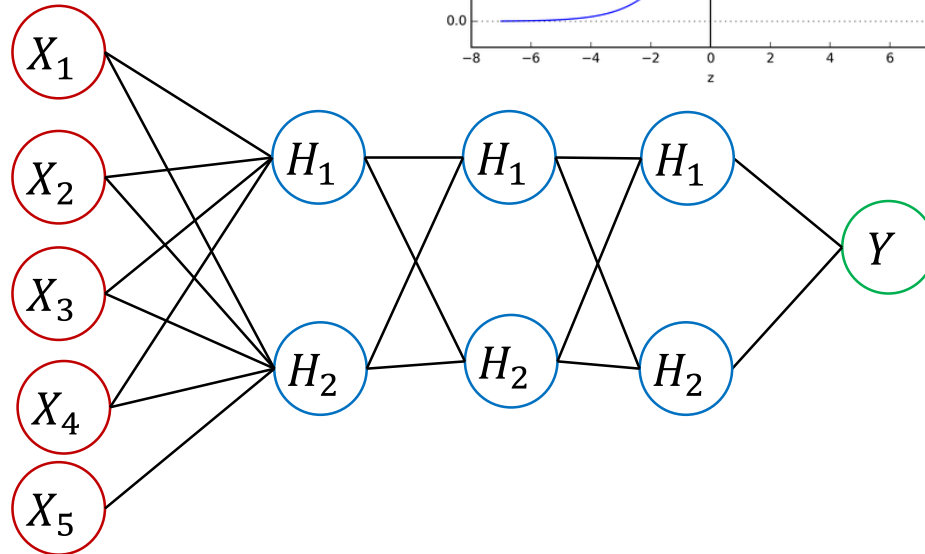
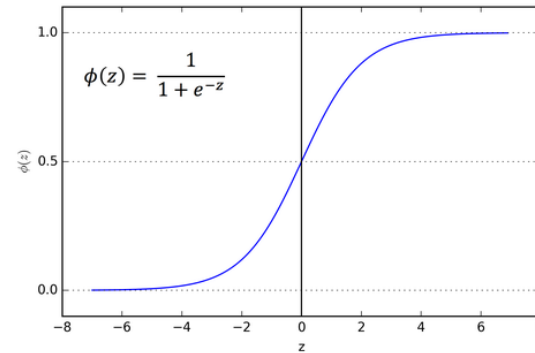
X(Input)

⋮	Sen3	Sen2	Sen1

word2vec 이용 문장평균

Bag of word – TF/TF-IDF

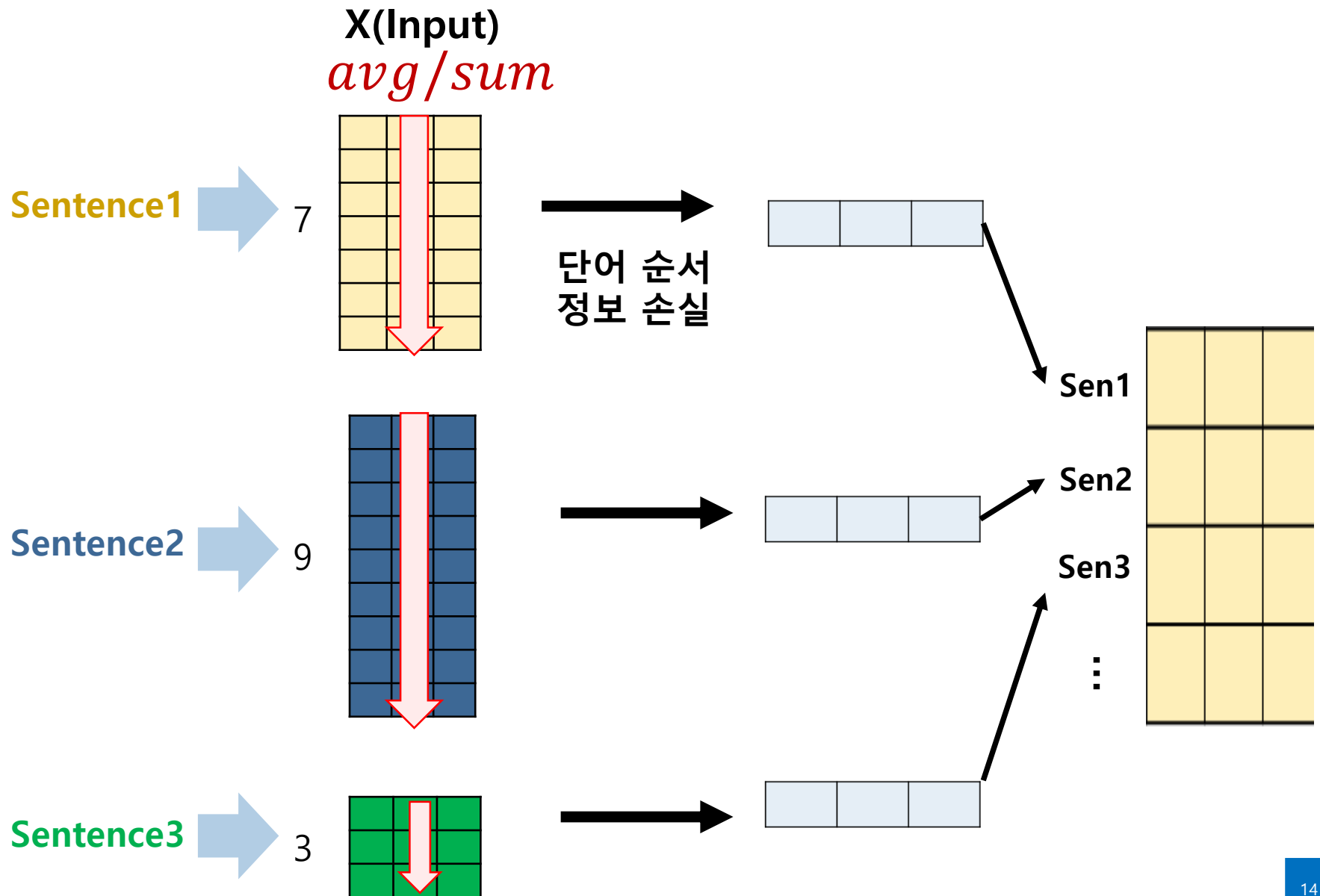
$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}} \quad \dots \text{sigmoid function}$$



0(공)

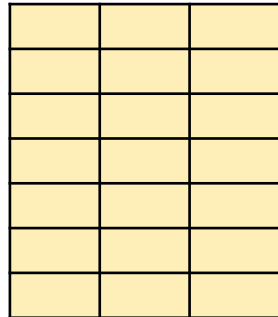
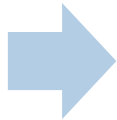
1(부)

Word2Vec – Text

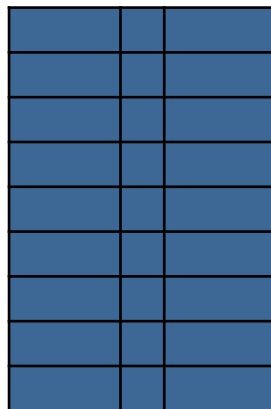
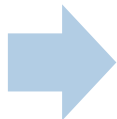


Convolutional Neural Networks (CNN) 활용

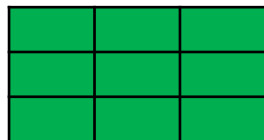
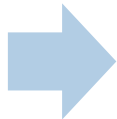
Sentence1



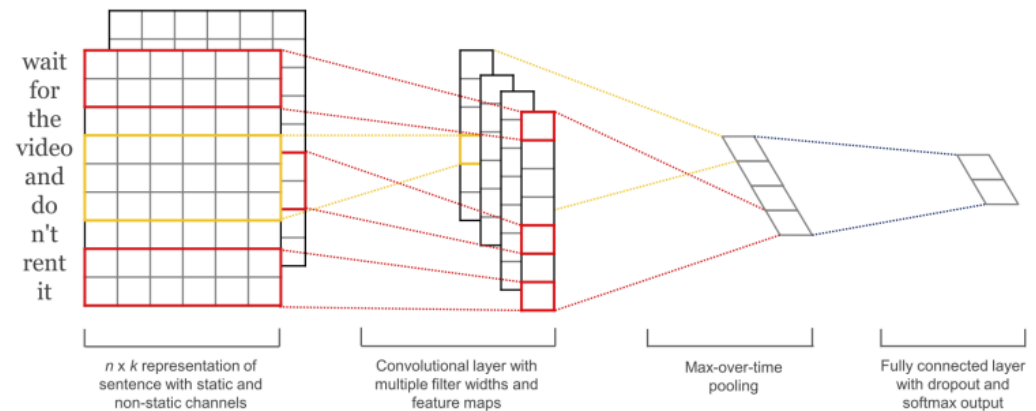
Sentence2



Sentence3

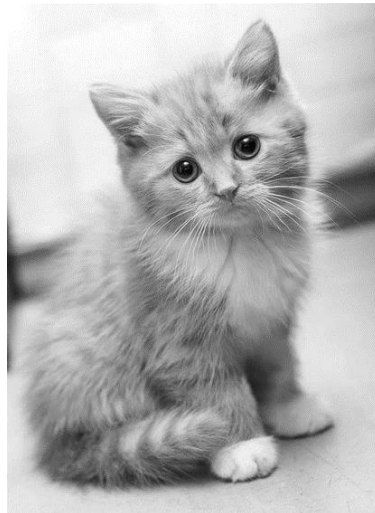


CNN for Sentence Classification
Yoon Kim (2014)

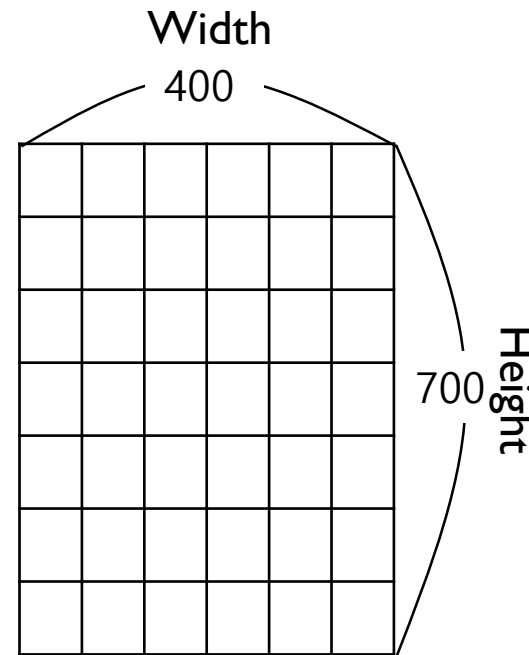


Convolutional Neural Networks (CNN) for Image

- 흑백 이미지는 2차원의 행렬로 표현됨 (Width, Height), 행렬 원소 $\rightarrow$ 픽셀
- 행렬은 흑백의 톤(짙고 옅은 정도)을 실수 값으로 표현
- 8 bit: 1~256 / 16 bit: 1~65,536

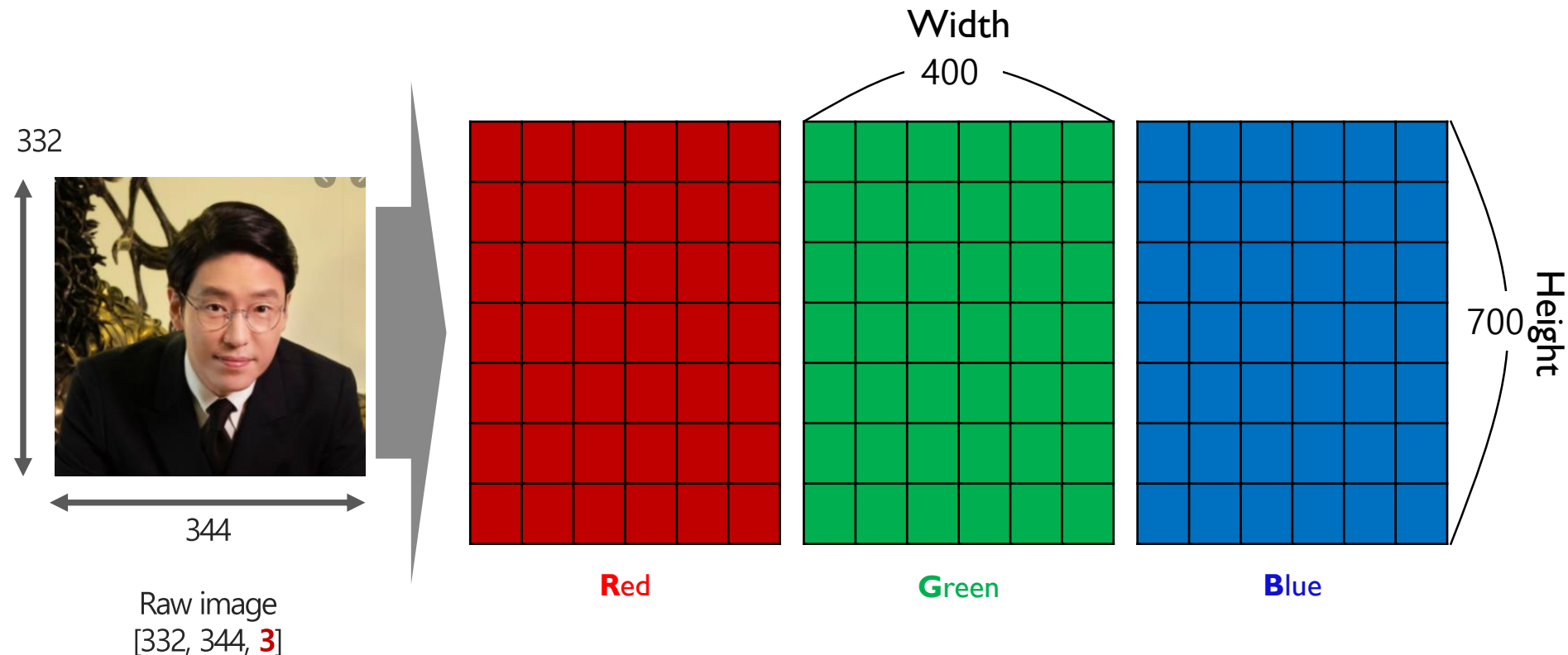


이미지
 400×700

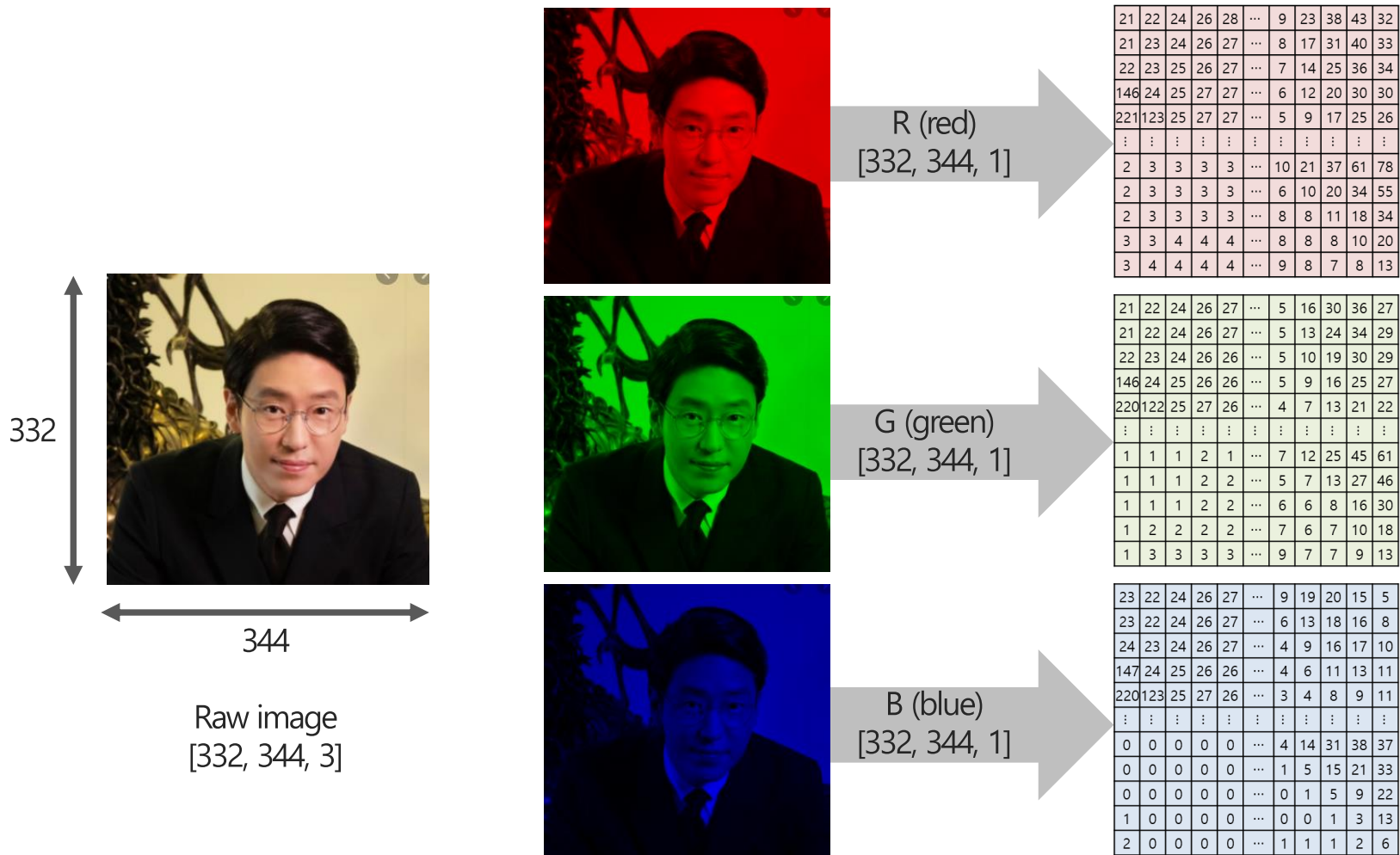


Convolutional Neural Networks (CNN) for Image

- 컬러 이미지는 3차원의 Tensor로 표현됨: Width, Height, RGB (Red, Green, Blue)
- 각각의 행렬은 색의 톤을 실수 값으로 표현
- 8 bit: 1~256 / 16 bit: 1~65,536



Convolutional Neural Networks (CNN) for Image



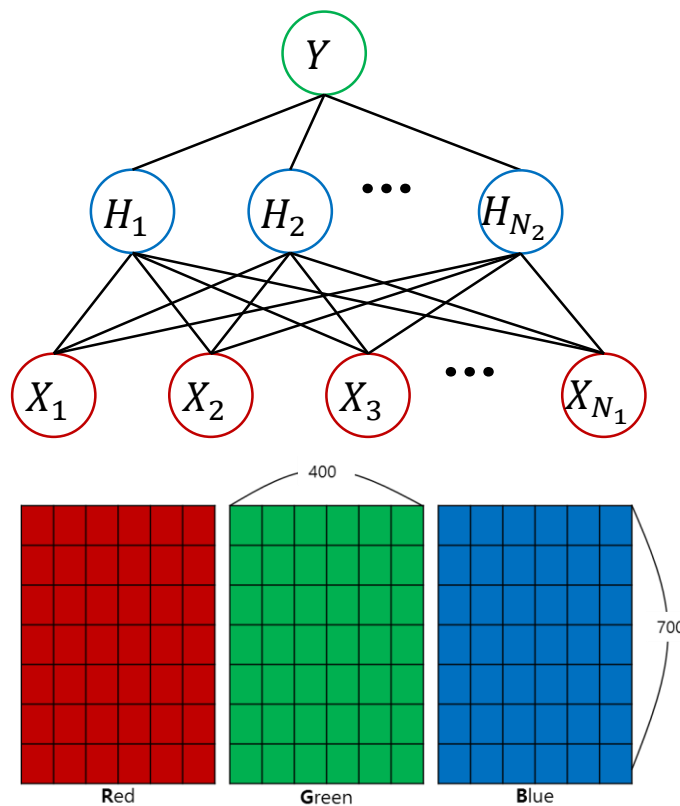
Convolutional Neural Networks (CNN) for Image



120	121	116	111	108	109	111	112	115	117	118	119	115	107	101	99	98	96	97	97	96	94	94	97	100	105	115	126	135	144	153	161		
136	129	122	121	116	115	119	125	128	133	134	132	131	133	131	129	124	119	115	111	111	110	107	106	107	108	111	119	128	137	143	151	159	
141	137	135	130	126	131	135	139	144	147	149	148	148	148	150	148	143	138	132	126	121	119	117	115	118	120	121	128	135	141	147	151	159	
145	144	142	140	142	149	152	153	156	157	158	159	159	160	160	155	155	151	147	137	130	126	123	122	127	130	132	138	142	147	151	155	159	
146	147	144	142	148	152	155	156	156	156	157	161	162	163	164	164	162	160	157	148	140	137	131	127	132	137	140	143	150	154	157	160	161	
149	141	137	135	138	144	149	150	152	154	157	163	166	167	170	174	178	184	188	187	186	179	173	170	164	158	157	155	154	158	162	165	170	171
134	132	127	129	135	141	144	146	151	156	161	166	170	174	178	184	188	187	186	179	173	170	164	158	157	155	154	158	162	165	170	171	172	
139	133	127	125	127	130	136	138	143	146	150	156	163	170	171	168	172	179	186	184	185	183	178	173	172	170	168	170	173	175	179	178	176	
139	131	124	123	124	127	133	139	142	146	152	159	163	168	176	179	180	182	186	186	183	185	181	179	180	185	183	179	180	181	183	185	188	
146	126	110	119	130	138	145	151	159	161	162	155	142	136	137	142	153	167	165	202	216	219	209	200	198	195	189	194	192	188	193	192	187	
152	129	126	135	141	145	146	144	139	130	110	103	89	79	74	75	81	82	110	126	142	161	184	193	194	195	198	197	196	194	192	195	189	
118	122	131	137	139	144	127	121	110	95	83	75	64	48	41	39	41	44	71	105	125	116	138	153	170	182	190	199	206	203	191	192	192	
116	119	126	123	117	112	105	86	82	89	89	71	46	35	27	20	25	30	64	79	108	108	107	116	116	138	169	194	208	207	190	185	179	
117	117	119	119	116	102	85	73	105	134	126	100	65	42	27	20	25	30	64	79	108	108	107	116	116	138	169	194	208	207	190	185	179	
120	122	123	124	125	120	108	100	83	108	133	137	137	79	40	34	26	38	35	49	85	151	165	127	104	107	114	139	175	200	207	203	194	174
129	131	133	131	127	131	132	134	135	133	135	142	80	59	49	42	41	48	63	108	175	200	172	122	98	107	113	148	185	201	201	196	174	
138	139	143	139	134	139	151	160	165	169	170	165	148	110	78	65	60	61	71	92	146	191	206	199	163	152	83	94	122	168	198	196	177	
154	155	153	149	146	154	162	169	175	184	192	190	176	147	124	113	111	120	149	186	204	212	206	191	156	123	107	114	158	196	195	192	179	
160	167	169	167	164	158	160	166	169	172	180	189	194	199	196	193	190	186	186	186	205	211	208	201	196	192	191	187	156	171	193	192	187	168
173	175	177	177	170	166	166	171	175	182	186	189	192	194	201	206	209	210	216	214	213	210	208	205	207	208	210	205	193	179	184	188	153	
179	181	183	183	178	172	170	173	178	185	193	196	198	203	207	210	213	218	218	216	216	212	205	200	196	197	192	179	167	162	159	142	140	
184	184	186	187	185	182	179	177	174	176	183	189	193	196	202	207	212	214	216	216	216	214	213	204	190	176	167	165	169	165	150	141	140	
187	189	191	192	191	188	184	179	174	175	179	183	187	191	196	200	205	207	208	207	206	203	204	203	193	179	162	151	157	171	157	153	156	
191	193	195	196	197	196	194	190	185	182	181	182	186	192	195	198	200	196	195	194	195	194	185	170	155	148	161	162	158	165	160	146	146	
194	195	196	201	202	202	201	199	192	187	182	176	173	172	174	176	177	180	184	186	190	195	197	197	193	180	164	159	173	150	130	141	140	
196	196	199	202	205	208	207	204	198	193	188	180	174	175	179	181	185	191	197	200	206	211	210	207	196	179	173	164	162	131	134	143	143	140
195	196	198	201	204	208	210	209	205	201	197	195	192	193	195	197	201	205	209	210	211	210	219	216	205	191	186	199	192	174	180	151	148	148
194	195	195	198	201	206	211	213	214	211	210	207	208	209	209	211	212	214	215	217	220	223	225	223	214	202	203	219	222	216	183	151	150	150
198	198	201	201	200	203	206	209	210	212	211	211	212	209	209	211	213	215	216	217	220	223	224	222	212	202	207	219	217	200	174	176	176	176
178	187	206	207	208	206	206	207	208	210	212	215	213	213	214	216	217	216	218	220	224	225	223	211	202	208	216	223	188	178	185	185	185	
173	188	206	212	213	212	210	209	211	213	216	217	218	219	221	223	224	225	227	226	218	205	198	197	170	154	180	188	184	172	172	172	172	
152	164	185	203	213	216	220	222	222	221	221	223	224	234	234	236	237	229	228	222	222	220	211	200	189	162	161	163	167	160	174	168	168	
193	196	188	186	183	178	222	221	222	223	224	225	225	227	227	227	227	229	236	239	240	241	200	181	156	163	164	167	163	175	169	171	171	
191	195	197	192	198	205	211	217	219	222	224	226	228	227	228	229	229	230	231	229	228	209	199	188	152	172	203	208	199	162	172	170	175	

Convolutional Neural Networks (CNN) for Image

- 각각의 픽셀을 하나의 변수로 간주하여 모델링
- $N_1 = 400(\text{width}) \times 700(\text{height}) \times 3(\text{RGB}) = 840,000$ 개
- N_2 의 크기도 변수의 수에 맞춰서 정함 $\rightarrow (840,000 \times N_2)$ 파라미터 $\rightarrow$ Input layer와 hidden layer 사이에 너무 많은 가중치 $\rightarrow$ **학습이 어렵다!**

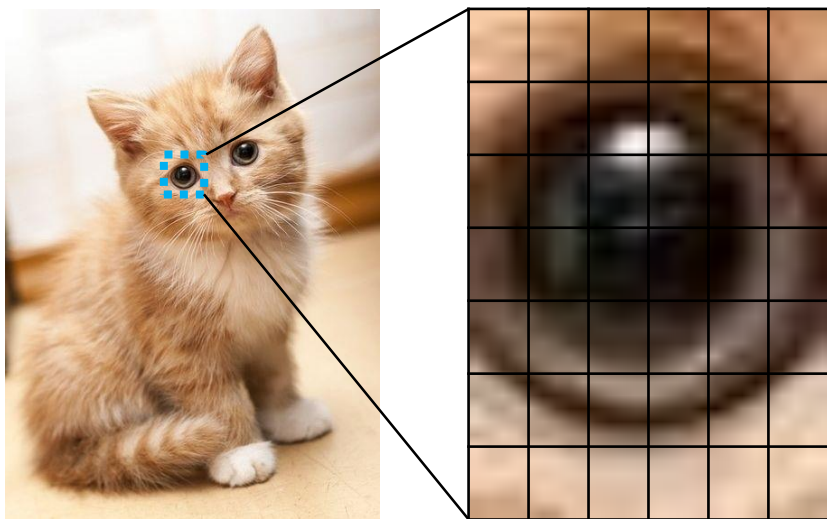


'데이터의 형상이 무시'

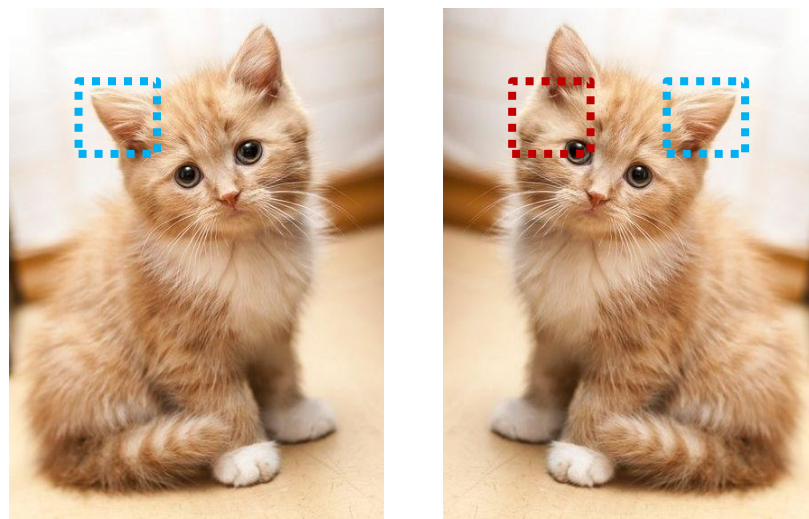
Convolutional Neural Networks (CNN) for Image

- 이미지의 경우 인접 변수(픽셀)간 높은 상관관계를 가짐 (spatially-local correlation)
- 이미지의 부분적 특성 (e.g., 눈, 귀)은 고정된 위치에 등장하지 않음 (invariant feature)
- 데이터의 특성을 반영한 모델을 구성하면 성능이 좋아지지 않을까?

Spatially-Local Correlation



Invariant Feature



Convolutional Filter + Neural Networks = CNN

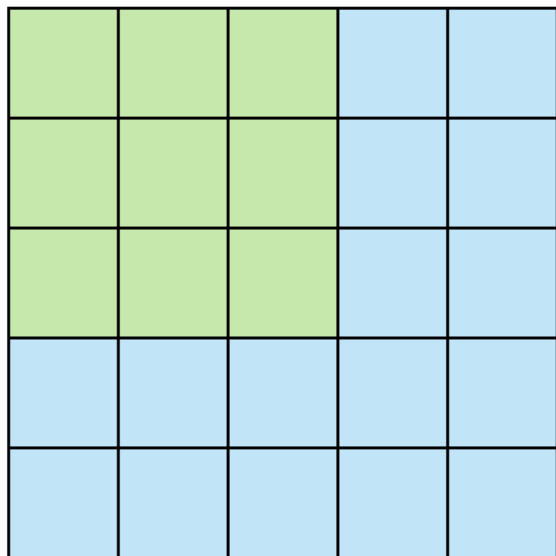
- Convolution with stride 1

w1	w2	w3
w3	w4	w5
w6	w7	w8

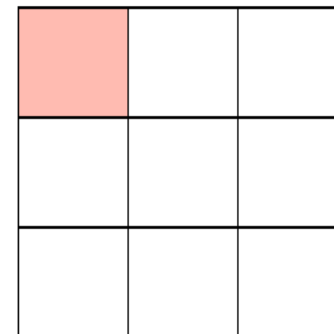
Filter (필터) or Kernel (커널)

Filter size

- Stride: 필터의 적용 위치 간격



Stride 1



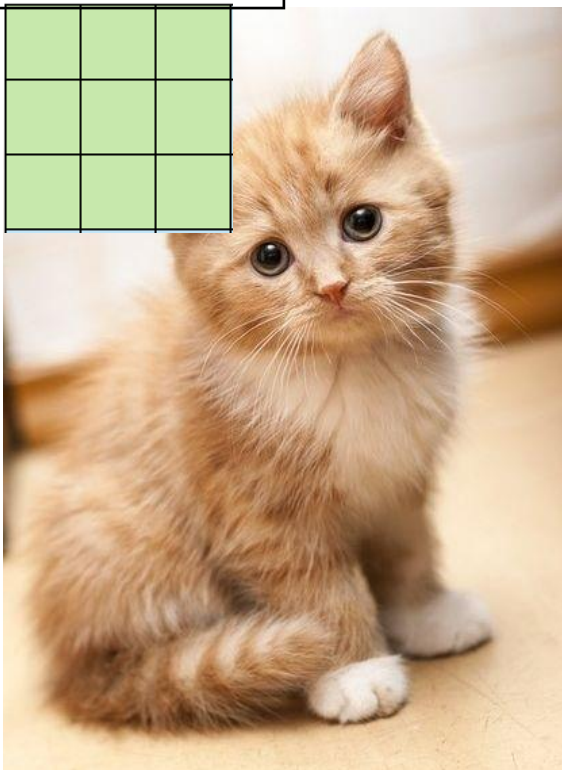
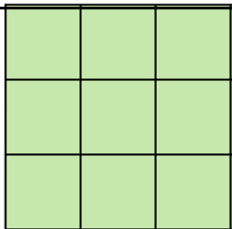
Feature Map

- Large Stride: 이미지의 특징을 놓칠 가능성 증가 → 계산효율 증대 / 성능 악화

Convolutional Filter + Neural Networks = CNN

Convolution with stride 1

27=> 3by3 3(RGB)



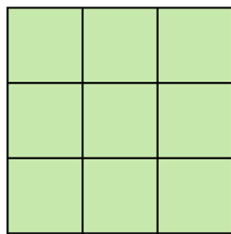
input data의 channel 수
와 filter의 channel 수는
동일

w1	w2	w3
w3	w4	w5
w6	w7	w8

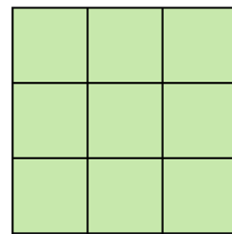
Filter (필터) or Kernel (커널)

Filter size, Channel size

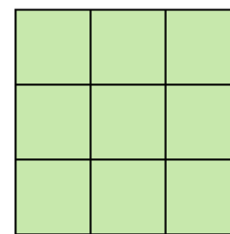
Filter - 귀



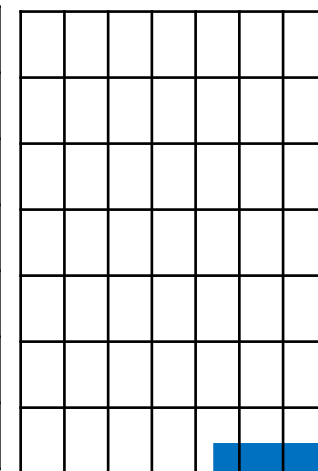
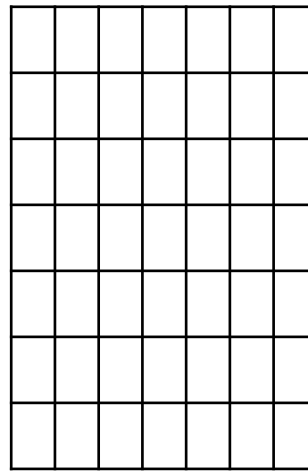
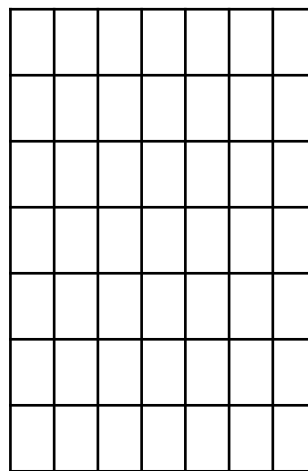
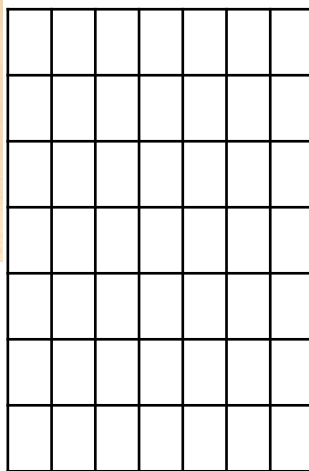
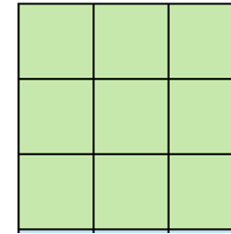
Filter - 눈



Filter - 수염



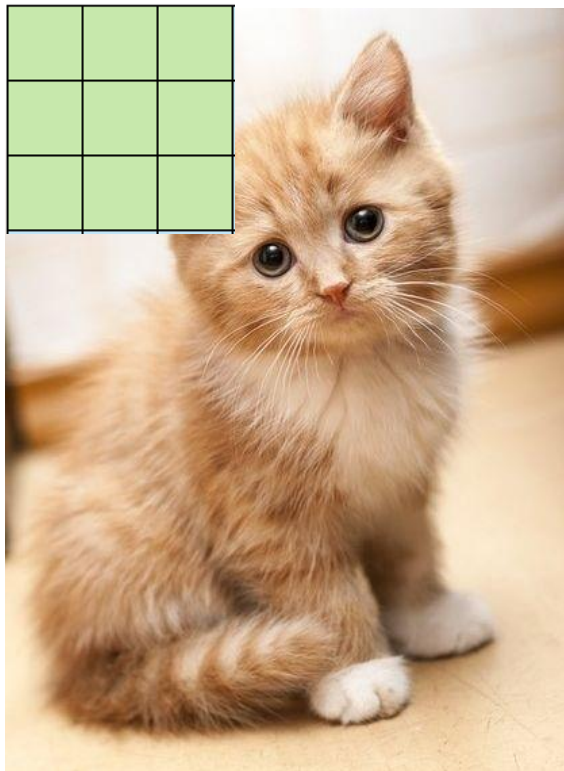
손



3by3 CNN 36 (, ,)

Convolutional Filter + Neural Networks = CNN

- Convolution with stride 2

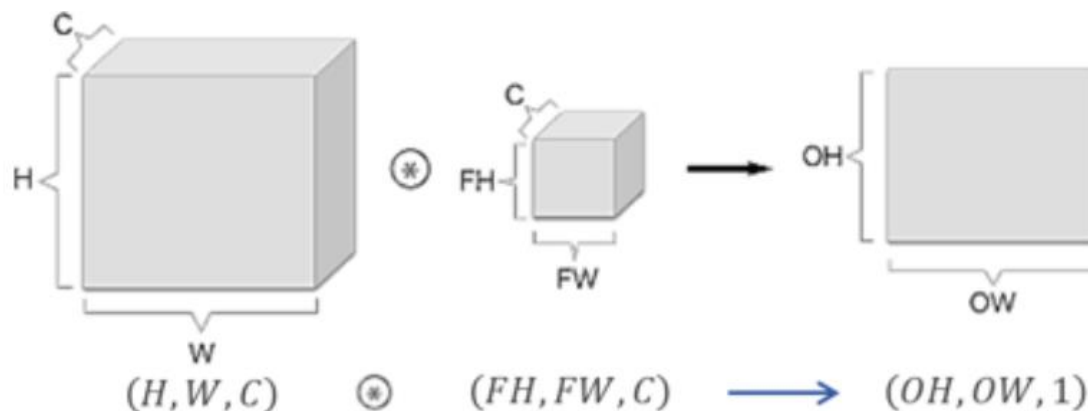
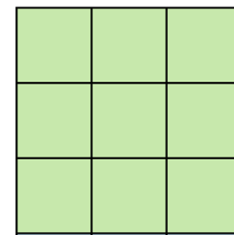


w1	w2	w3
w3	w4	w5
w6	w7	w8

Filter (필터) or Kernel (커널)

Filter size , Channel size

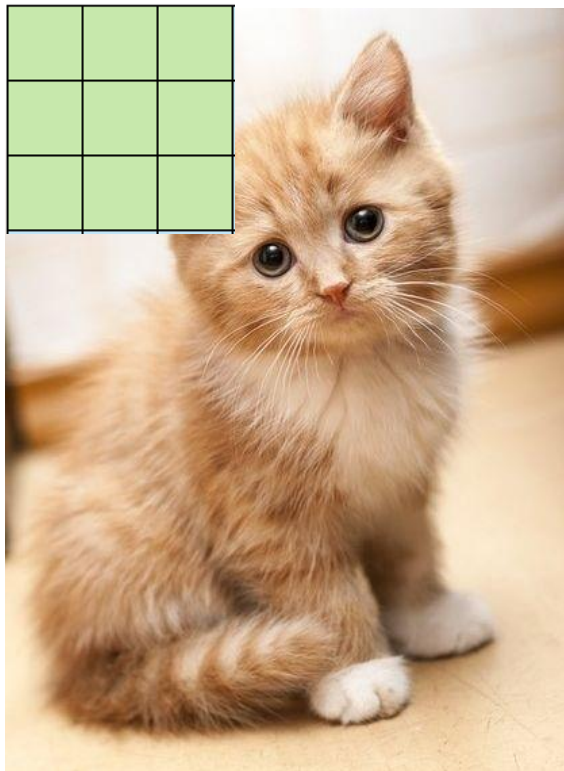
Filter –고양이 귀



- Stride: 필터의 적용 위치 간격

Convolutional Filter + Neural Networks = CNN

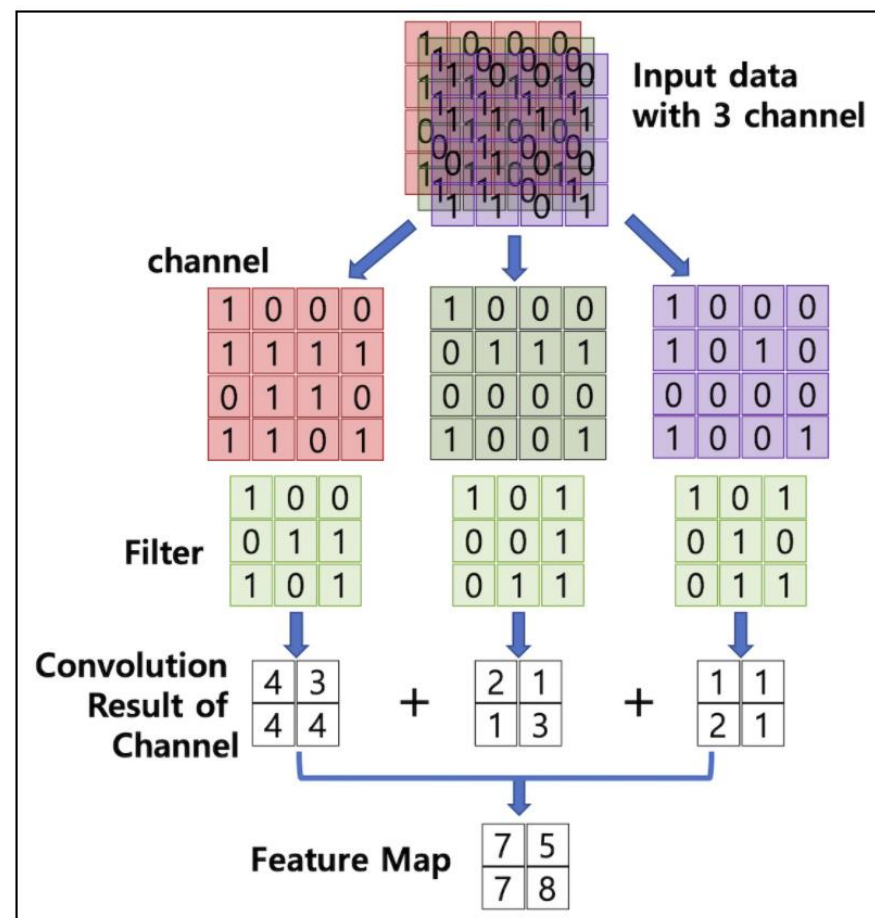
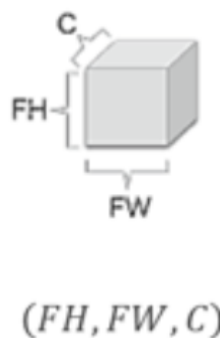
- Convolution with stride 2



w1	w2	w3
w3	w4	w5
w6	w7	w8

Filter (필터) or Kernel (커널)

Filter size , Channel size



- Stride: 필터의 적용 위치 간격

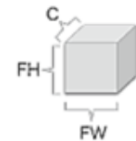
Convolutional Filter + Neural Networks = CNN

- Convolution with stride 2

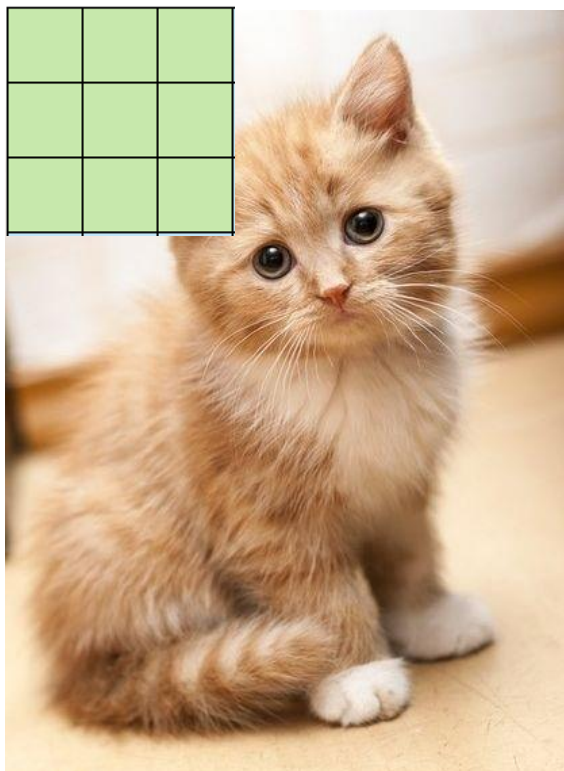
w1	w2	w3
w3	w4	w5
w6	w7	w8

Filter (필터) or Kernel (커널)

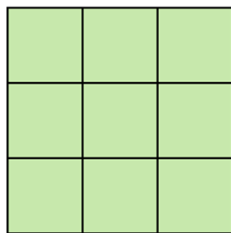
Filter size



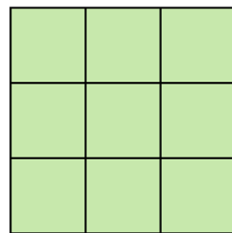
(FH, FW, C)



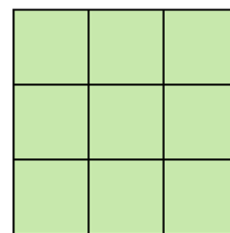
Filter - 귀



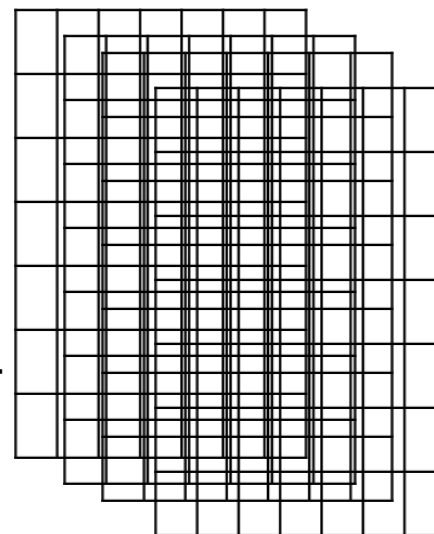
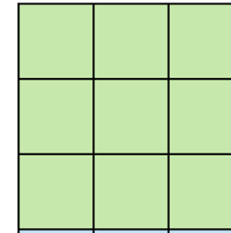
Filter - 눈



Filter - 수염

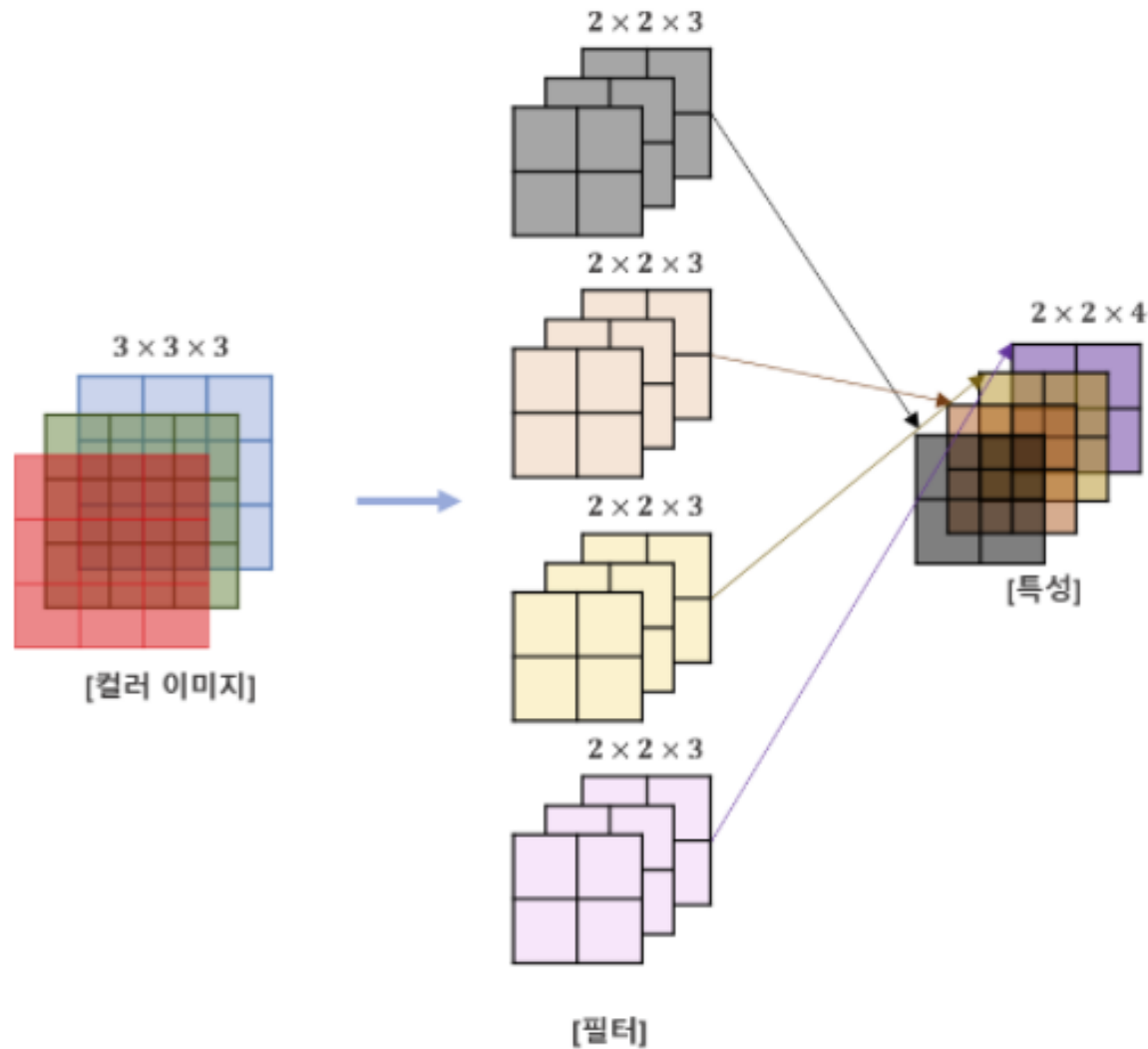


손



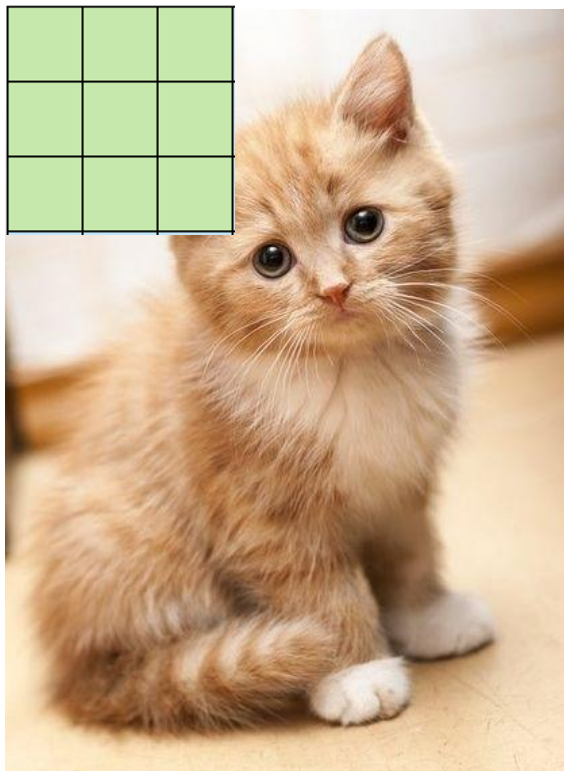
필터 수 = 아웃풋 채널 = 4

Convolutional Filter + Neural Networks = CNN



Convolutional Filter + Neural Networks = CNN

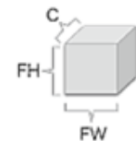
Convolution with stride 2



w1	w2	w3
w3	w4	w5
w6	w7	w8

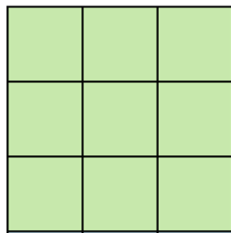
Filter (필터) or Kernel (커널)

Filter size

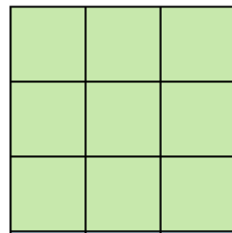


(FH, FW, C)

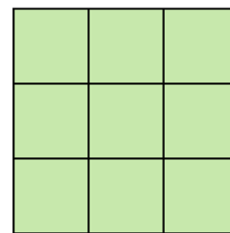
Filter -귀



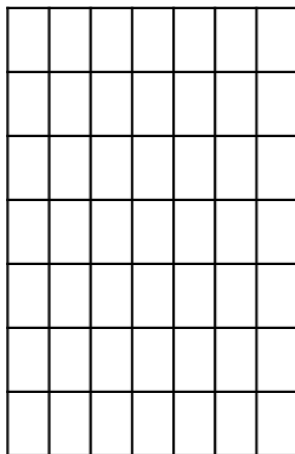
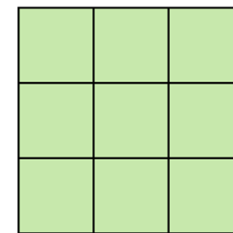
Filter -눈



Filter -수염



손

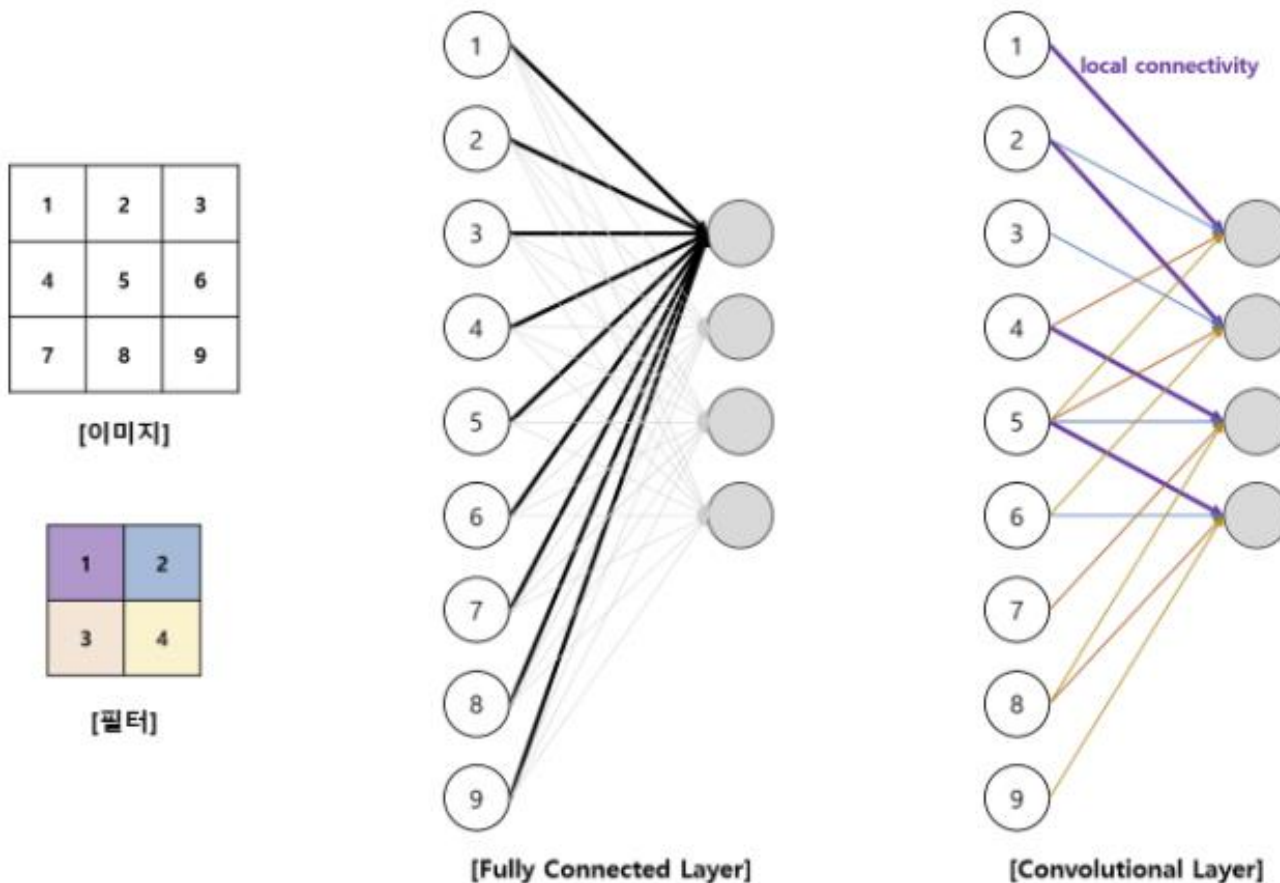


1. Spatially-Local Correlation

2. Invariant Feature

3. 파라미터 공유 -> 학습 파라미터 감소

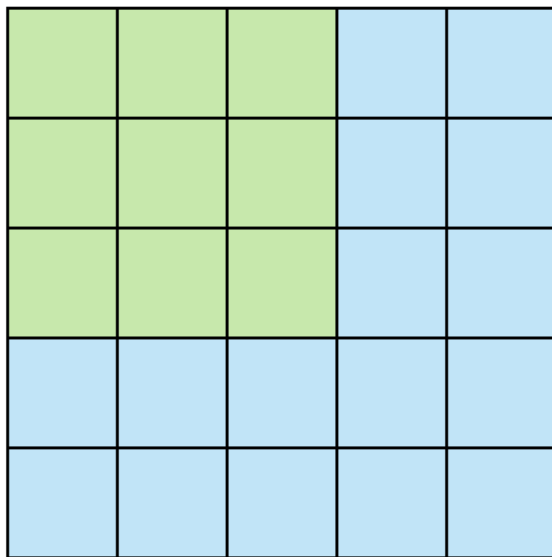
Convolutional Filter + Neural Networks = CNN



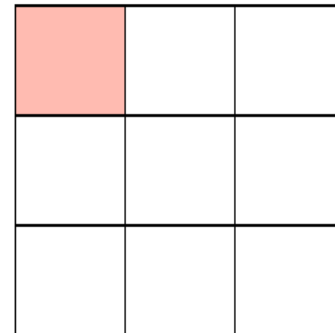
파라미터 공유 -> 학습 파라미터 감소

픽셀의 일부만 연결 (local connectivity)
가중치 공유

Convolutional Filter + Neural Networks = CNN



Stride 1



Feature Map

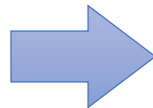
Convolutional Filter + Neural Networks = CNN

- Convolution 결과 이미지가 입력 이미지의 크기와 같으면 편리할 때가 많음
- 가장자리에 있는 픽셀들은 중앙에 위치한 픽셀들에 비해 Convolution 연산이 적게 수행됨
- 이를 위한 해결책으로 원 이미지 테두리에 0의 값을 갖는 픽셀 추가
- 테두리에 추가된 0값의 픽셀 → Pad, 테두리에 추가된 0값의 픽셀 추가 → Zero Padding

패딩(Padding)은 합성곱 연산을 수행하기 전

3	1	1	2	8	4
1	0	7	3	2	6
2	3	5	1	1	3
1	4	1	2	6	5
3	2	1	3	7	2
9	2	6	2	5	1

Padding
with size 1

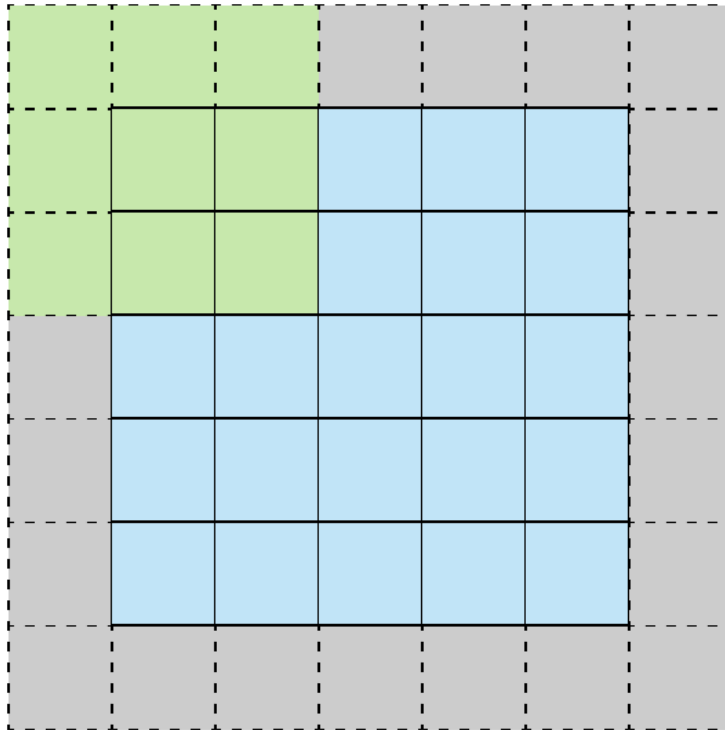


0	0	0	0	0	0	0	0
0	3	1	1	2	8	4	0
0	1	0	7	3	2	6	0
0	2	3	5	1	1	3	0
0	1	4	1	2	6	5	0
0	3	2	1	3	7	2	0
0	9	2	6	2	5	1	0
0	0	0	0	0	0	0	0

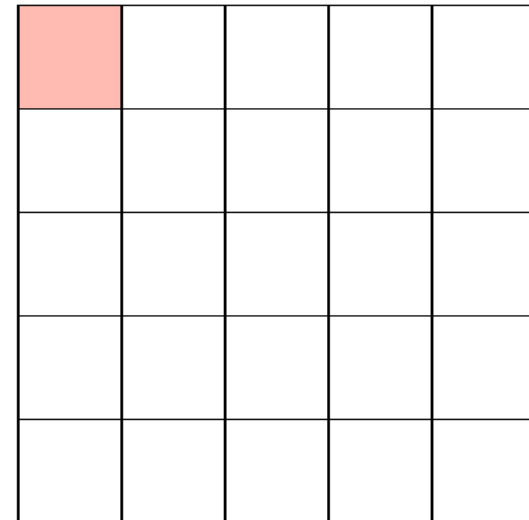
Convolutional Filter + Neural Networks = CNN

Padding

- 원 이미지 테두리에 0값의 픽셀 추가 → Padding (Zero-Padding)



Stride 1 with Padding

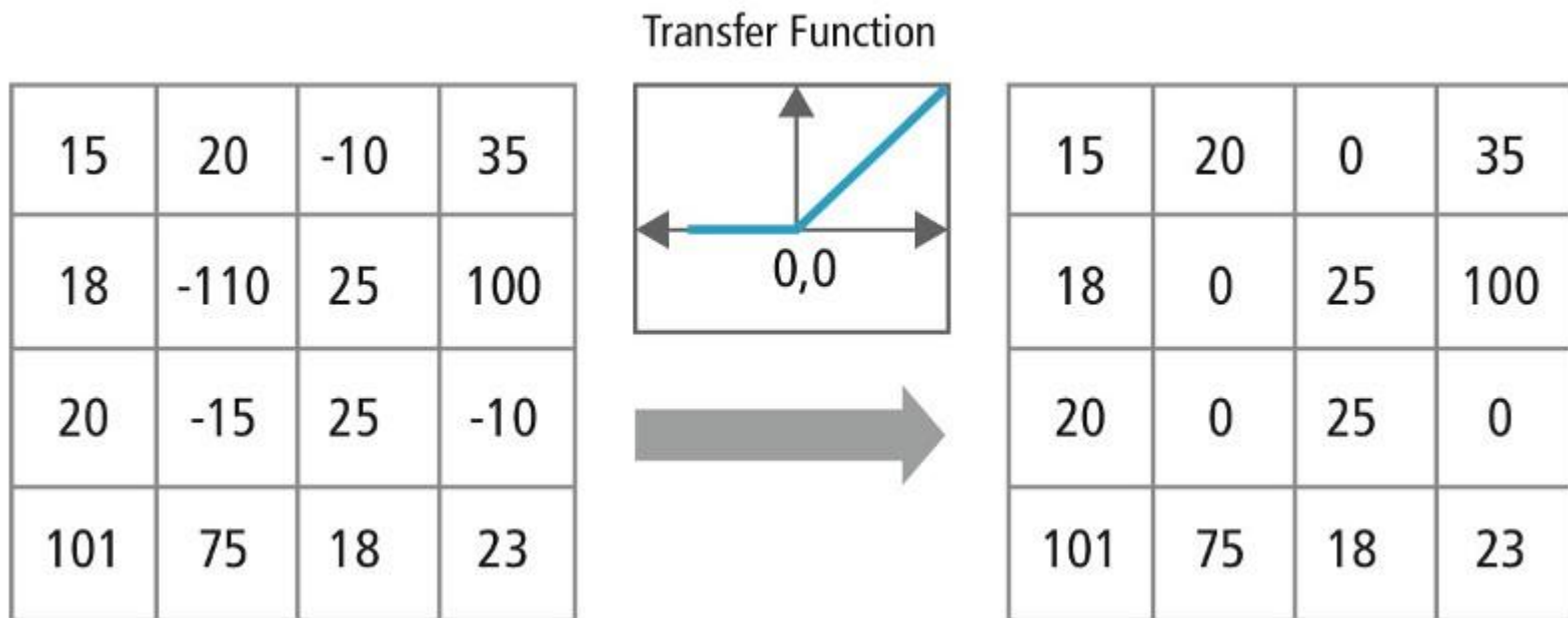


Feature Map

Convolutional Neural Networks (CNN) for Image

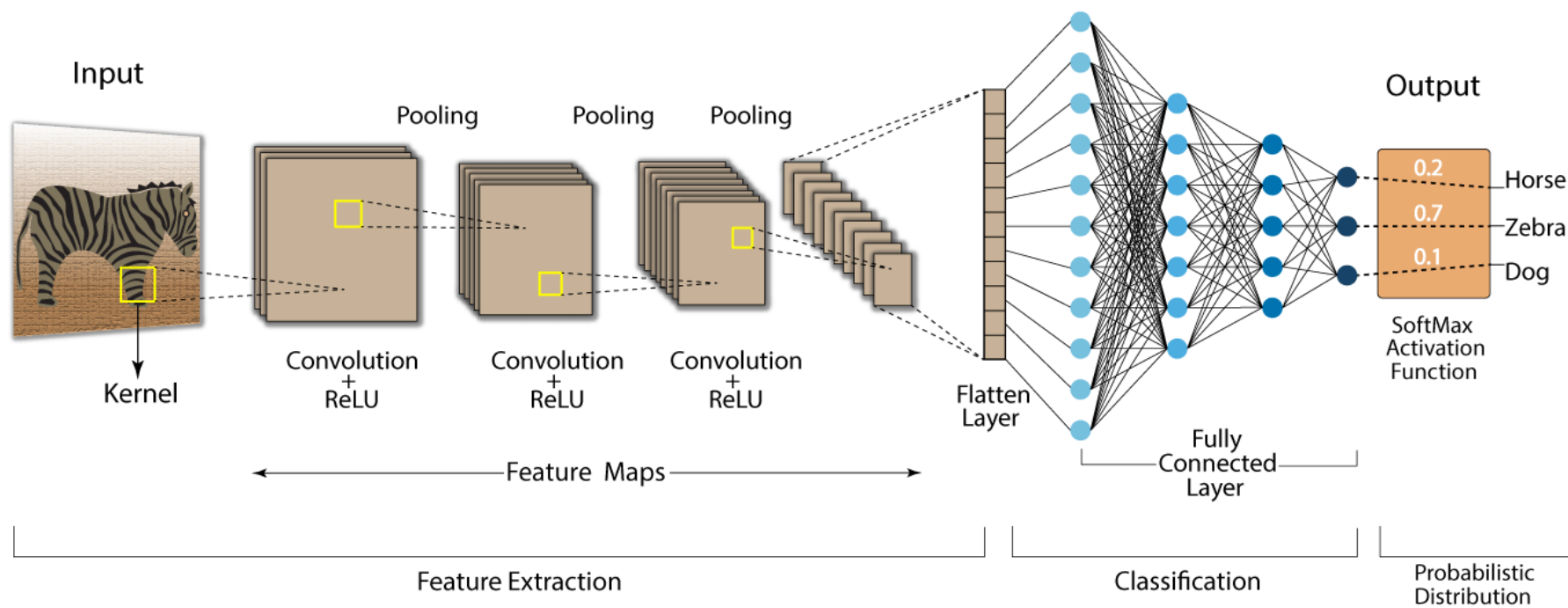
Activation function

- Convolution을 통해 학습된 값들의 비선형 변환
- Rectified Linear Unit (ReLU)를 주로 사용



Convolutional Neural Networks (CNN) for Image

Convolution Neural Network (CNN)

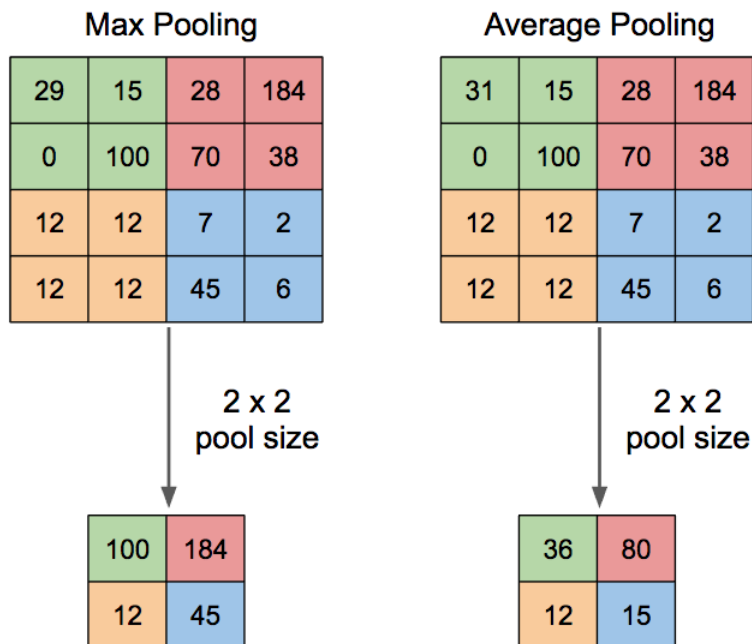


- 2D 혹은 3D구조를 유지하면서 학습
- 일반적인 CNN은 Convolution 연산, Activation 연산, Pooling 연산의 반복으로 구성됨 (Feature Learning, Feature representation)
- 일정 횟수 이상의 Feature Learning 과정 이후에는 Flatten 과정을 통해 이미지가 1차원의 벡터로 변환됨

Convolutional Filter + Neural Networks = CNN

Pooling

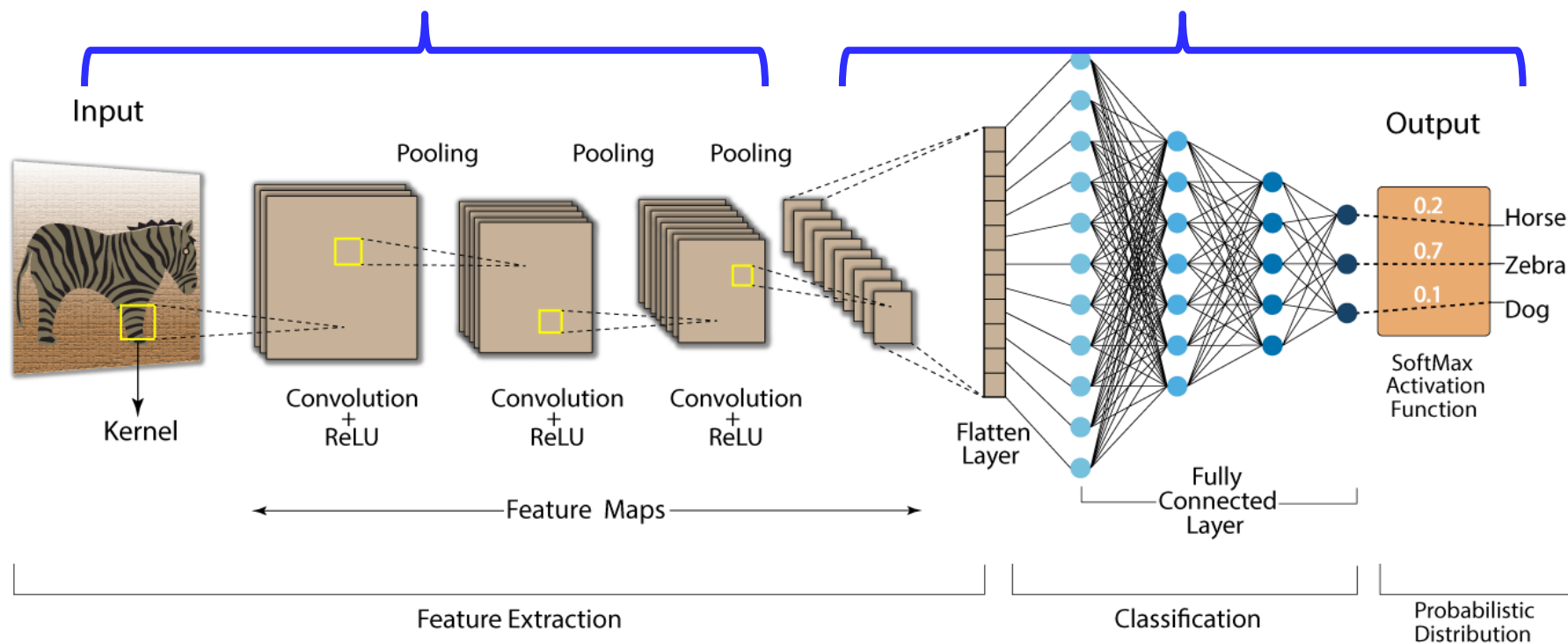
- Feature Map의 공간적 특징은 유지, 크기는 감소
- 일정 영역의 정보를 축약 → Pooling
- Max Pooling, Average Pooling (이미지 분석에서는 주로 Max Pooling)
- Pooling stride 크기: 주로 Pooling 크기와 동일하게 설정
- Pooling 시 padding은 보통 사용하지 않음



Convolutional Neural Networks (CNN) for Image

특징 추출: Representation Learning 분류: Fully connected layer

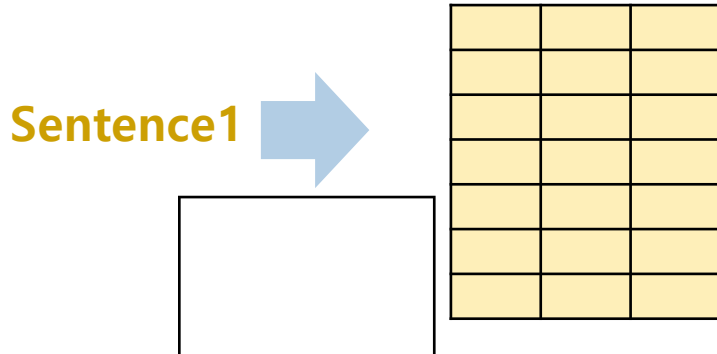
Convolution Neural Network (CNN)



- 2D 혹은 3D구조를 유지하면서 학습
- 일반적인 CNN은 Convolution 연산, Activation 연산, Pooling 연산의 반복으로 구성됨 (Feature Learning, Feature representation)
- 일정 횟수 이상의 Feature Learning 과정 이후에는 Flatten 과정을 통해 이미지가 1차원의 벡터로 변환됨

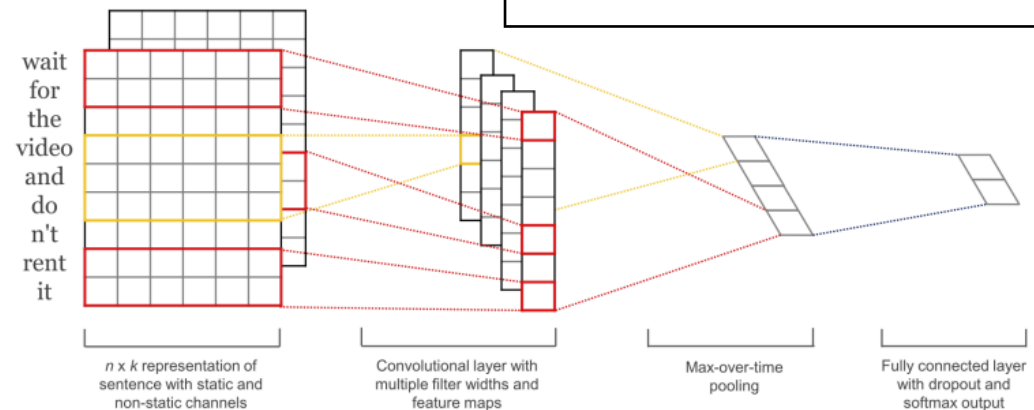
Convolutional Neural Networks (CNN) 활용

embedding 결과를 이미지 처럼 생각
2차원 모양의 channel=1



이미지의 크기가 동일?

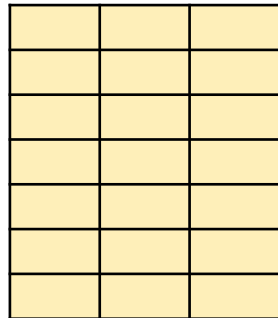
CNN for Sentence Classification
Yoon Kim (2014)



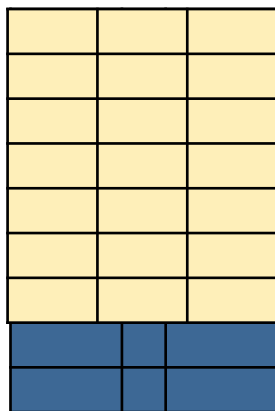
Convolutional Neural Networks (CNN) 활용

embedding 결과를 이미지 처럼 생각
2차원 모양의 channel=1

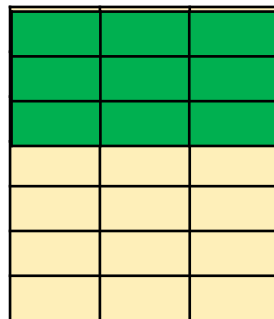
Sentence1 →



Sentence2 →



Sentence3 →

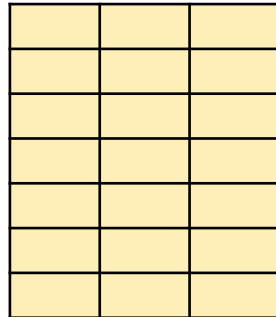
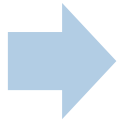


- 이미지의 크기가 동일?
Input Data에 대한 패딩이 필요

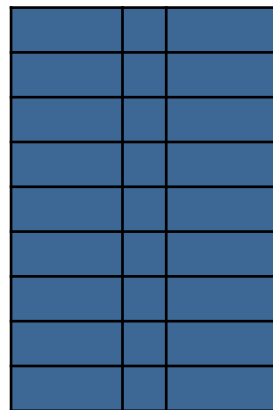
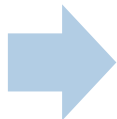
Convolutional Neural Networks (CNN) 활용

embedding 결과를 이미지 처럼 생각
2차원 모양의 channel=1

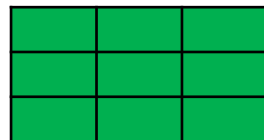
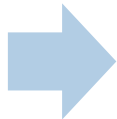
Sentence1



Sentence2



Sentence3



이미지의 크기가 동일?

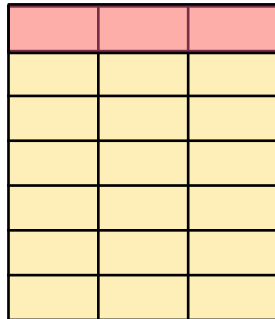
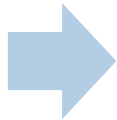
Input: n by k (by m) representation of sentence

- n: the number of words in a sentence (parameter)
 - ✓ Shorter sentences are zero-padded and longer sentences are trimmed
- k: word embedding dimensions
 - ✓ Pre-trained word embedding vectors can be used
 - ✓ These vectors can be static (not updated during the training) or non-static (finetuned for the task-specific corpus)
- Multi-channel input is also possible → an input sentence becomes a tensor
 - ✓ Pre-trained word embedding by Word2Vec (Static)
 - ✓ Pre-trained word embedding by Word2Vec (Non-static)
 - ✓ Pre-trained word embedding by Glove (Static)
 - ✓ 실험적 결과에서 성능적 차이 크게 없었음

Convolutional Neural Networks (CNN) 활용

embedding 결과를 이미지 처럼 생각
2차원 모양의 channel=1

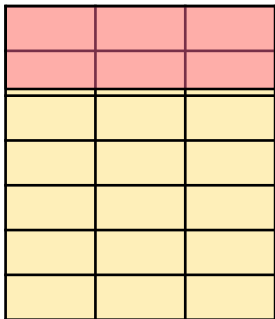
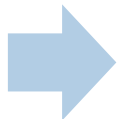
Sentence1



- 필터(kernel size)??

기존은 2D kernel
1D kernel

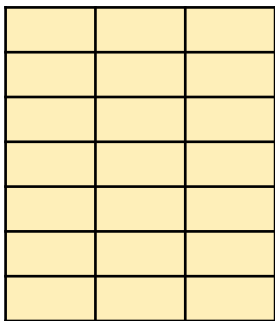
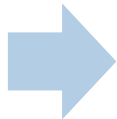
Sentence2



- A squared convolution is common for image processing, but a rectangular convolution with the width of k is used for text processing

- Convolution stride is usually set to 1

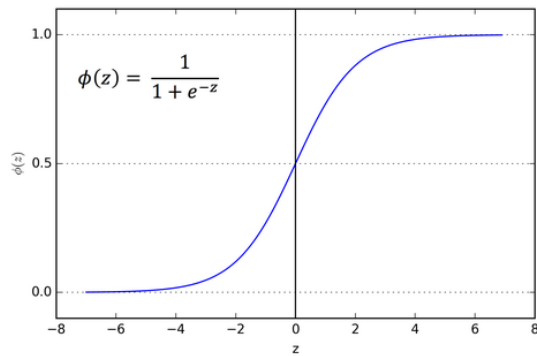
Sentence3



- The larger the height, the more words are considered by the convolution at a single time

Convolutional Neural Networks (CNN) 활용

$\text{sigmoid}(x) = \frac{1}{1+e^{-x}}$... sigmoid function



- Classification: cross entropy
- $\sum_i L(Y_i, f(X_i; w))$
- $L(y_c, \hat{y}_c) = -\sum_c y_c \log \hat{y}_c$ (y_c :실제값, $\hat{y}_c$:예측값)

